

- Lenguaje $\implies$ Comunicación
- Lenguaje de programación $\implies$ Comunicación con la máquina
- Lenguaje hablado y lenguaje escrito
- Lenguaje escrito $\implies$ Formalismos $\implies$ Lenguajes formales

- Sistemas de escrituras independientes del medio:
 - Sumerios
 - Egipcios
 - Mayas



Figura: Ejemplo de escritura egipcia, tomado de enciclopedia britannica

Evolución de los alfabetos: Pictogramas a Letras.

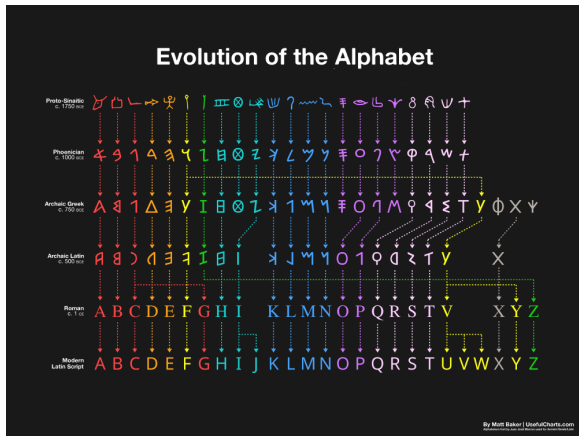


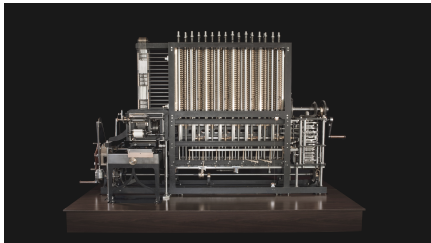
Figura: Evolucion de pictogramas a letras, fuente:
<https://usefulcharts.com/blogs/charts/evolution-of-the-english-alphabet>

La **programación** se define como una actividad general del hombre, que significa la acción de extender o cambiar la funcionalidad de un sistema[VanRoy].

- Programar es decirle a un computador (o a alguna máquina) como realizar su trabajo.
- La programación es una actividad de amplio espectro
- La programación de sistemas de software consta de dos partes esenciales: la ciencia y la tecnología.

- Un **lenguaje de programación** es un lenguaje artificial diseñado para expresar computaciones que pueden ser llevadas a cabo por una máquina.
- Conjunto de símbolos y reglas sintácticas y semánticas.

- Charles Babagge (Máquina Analítica) y Ada Lovelace (Primera programadora) (Mediados del siglo XIX).



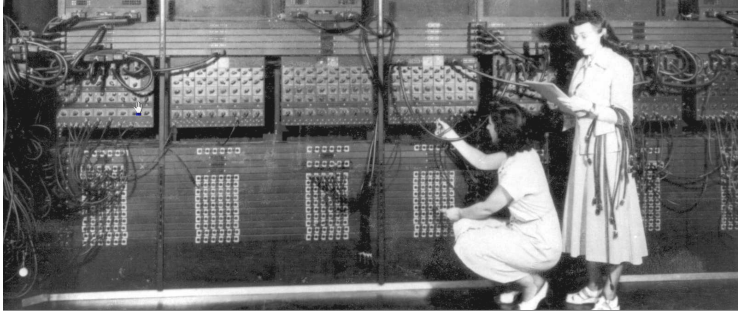
- Programa escrito por Ada Lovelace

Diagram for the computation by the Engine of the Numbers of Bernoulli. See Note G. (page 722 *et seq.*)

[illegible]

- Herman Hollerith se percató de que podía codificar la información en tarjetas perforadas cuando observó a los conductores de trenes que identificaban a los pasajeros según el orificio que hacían en su respectivo ticket. En 1890 Hollerith codificó los datos del censo en tarjetas perforadas.
- En la década de 1920 los cálculos numéricos estaban basados en los números decimales. Con el paso del tiempo, se dieron cuenta de que la lógica podía ser representada con números, no sólo con palabras.

- En 1943 se crea el sistema de codificación de ENIAC la primera computadora de propósito general.



-24 to C	- b_1	-	-	1	00011	010
c to 26			- b_1	2	01011	110
-26 to C	b_1			3	01011	010
c to 27			- b_1 b_1	4	11011	110
-23 to C	a	T_{n-1}	- b_n b_n	5	11101	010
sub. 27	$a - b_n$			6	11011	001
Test				7	-	011
Add 20 to bl.				8	00101	100
sub. 26	r_n			9	01011	001
c to 25		r_n		10	10011	110
-25 to C				11	10011	010
Test				12	-	011
Stop	0	0	- b_n b_n	13		111
-26 to C	b_n	r_n	- b_n b_n	14	01011	010
sub. 21	b_{n-1}			15	10101	001
c to 27	b_{n+1}		b_{n+1}	16	11011	110
-27 to C	- b_{n+1}			17	11011	010
c to 26			- b_{n+1}	18	01011	110
22 to bl.		r_n	- b_{n+1} b_{n+1}	19	01101	000

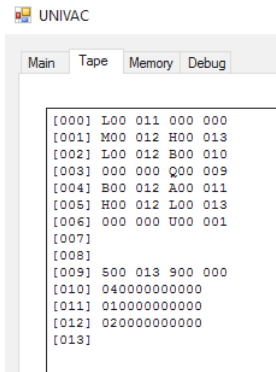
or 000

20	-3	10111 etc	23	-a	25	init.	final
21	1	10000				-	$r_n (=0)$

- Short Code de John Mauchly en 1949 (BINAC y UNIVAC I).
Por ejemplo la expresión: $a = \frac{b+c}{b*c}$ se computa así:

x3 = (x1 + y1) / x1 * y1	substitute variables
x3 03 09 x1 07 y1 02 04 x1 y1	substitute operators and parentheses
	Note multiplication is represented
	by juxtaposition.
07 y1 02 04 x1 y1	group into 12-byte words.
00 00 x3 03 09 x1	

- A-0, A-1, A-2 entre 1951 y 1953 (UNIVAC). El siguiente programa calcula el factorial de un número almacenado en un registro.



Mayor información en:

<http://museo.inf.upv.es/en/univac/> así mismo se tiene
un simulador disponible en:

<http://www.historicsimulations.com/univac.html>

- Fortran (FORmula TRANslator) por John Backus et al. en 1953.

```
C AREA OF A TRIANGLE — HERON'S FORMULA
C INPUT — CARD READER UNIT 5, INTEGER INPUT
C OUTPUT — LINE PRINTER UNIT 6, REAL OUTPUT
C INPUT ERROR DISPLAY ERROR OUTPUT CODE 1 IN JOB CONTROL LISTING
      INTEGER A,B,C
      READ(5,501) A,B,C
501  FORMAT(3I5)
      IF(A.EQ.0 .OR. B.EQ.0 .OR. C.EQ.0) STOP 1
      S = (A + B + C) / 2.0
      AREA = SQRT( S * (S - A) * (S - B) * (S - C) )
      WRITE(6,601) A,B,C,AREA
601  FORMAT(4H A= ,I5,5H B= ,I5,5H C= ,I5,8H AREA= ,F10.2,
           $13H SQUARE UNITS)
      STOP
      END
```

- LISP (LISt Processor) por John McCarthy et al. en 1958.

```
(defun averagenum (n1 n2 n3 n4)
  (/ (+ n1 n2 n3 n4) 4)
)
(write(averagenum 10 20 30 40))
```

- COBOL (COmmon Business Oriented Language) por Grace Hopper en 1959.

```
DATA DIVISION.
```

```
WORKING-STORAGE SECTION.
```

```
01  Num1                      PIC 9  VALUE ZEROS.  
01  Num2                      PIC 9  VALUE ZEROS.  
01  Result                   PIC 99 VALUE ZEROS.
```

```
PROCEDURE DIVISION.
```

```
    DISPLAY "Enter first number (1 digit) : " WITH NO ADVANCING.  
    ACCEPT Num1.  
    DISPLAY "Enter second number (1 digit) : " WITH NO ADVANCING.  
    ACCEPT Num2.  
    MULTIPLY Num1 BY Num2 GIVING Result.  
    DISPLAY "Result is = ", Result.  
    STOP RUN.
```

■ ALGOL (ALGOrithmic Language) 60 en 1960.

```
procedure Absmax(a) Size:(n, m) Result:(y) Subscripts:(i, k);  
  value n, m; array a; integer n, m, i, k; real y;  
begin  
  integer p, q;  
  y := 0; i := k := 1;  
  for p := 1 step 1 until n do  
    for q := 1 step 1 until m do  
      if abs(a[p, q]) > y then  
        begin y := abs(a[p, q]);  
              i := p; k := q  
        end  
      end  
end Absmax
```

- APL (A Programming Language) por Kenneth Iverson(IBM)
<https://tryapl.org/>
- Simula por Ole Johan Dahl y Kristen Nygaard y SNOBOL (StriNg Oriented symBolic Language) por los Laboratorios Bell en 1962. Este es el primer lenguaje orientado a objetos

```
Begin
  OutText ("Hello, World!");
  Outimage;
End;
Begin
  Class Glyph;
    Virtual: Procedure print Is Procedure print;;
  Begin
  End;
```

- CPL (Combined Programming Language) en 1963. Está basado en Algol y simplifica la creación de programas en computación científica.

```
rec function Fact1[x] = (x = 0) -> 1, xFact1[x - 1]
```


- BASIC (Beginners All-purpose Symbolic Instruction Code) por Kurtz y PL/1 (*Programming Language 1*) de IBM en 1964.

```
PROGRAM series
! add the first 100 terms of a simple series
! True BASIC automatically initializes variables to zero,
! but other languages might not.
LET sum = 0
FOR n = 1 to 100
    LET sum = sum + 1/(n*n)
    PRINT n,sum
NEXT n
END
```

Ejemplo en PL/1

```
Test: procedure options(main);

    declare My_String char(20) varying initialize('Hola, mundo!');

    put skip list(My_String);

end Test;
```

- BCPL (Basic Combined Programming Language) por Martin Richards en 1967.
- ALGOL 68 y Logo por Danny Bobrow, Wally Feurzeig y Seymour Papert en 1968. Logo es un lenguaje funcional y estructurado, de muy fácil aprendizaje. Se utiliza para la enseñanza de la programación en niños y jóvenes.
<https://www.calormen.com/jslogo/>
- C por Dennis Ritchie y Ken Thompson entre 1969 y 1973. Influido por Algol 68 (Lenguaje A) y BCPL (Lenguaje B).
- Pascal por Wirth y SmallTalk en 1970. Pascal es estáticamente y fuertemente tipado. SmallTalk es de tipado dinámico.

```
Program Lesson1_Program1;  
Begin  
    Write('Hello World. Prepare to learn PASCAL!!');  
    Readln;  
End.
```

```
'Hello World!' displayNI  
1 to: 20 do: [:x | x printNI ] !  
  
x := Array new: 20.  
x printNI.  
(x at: 1) printNI.  
x at: 1 put: 99 .  
(x at: 1) printNI .
```

- Prolog (PROgrammation en LOGique) por Colmerauer, Roussel, y Kowalski en 1972.
 - Base del conocimiento:

```
likes(mary, food).  
likes(mary, wine).  
likes(john, wine).  
likes(john, mary).
```

■ Consultas:

```
likes (mary , food) .  
Yes  
likes (john , food) .  
No  
likes (X, wine) .  
mary  
john  
No
```

■ ML (Meta Language) por Robin Milner en 1973.

```
fun reverse [] = []  
  | reverse (h :: t) = reverse t @ [h];  
  
fun concat_space s = s ^ " ";  
  
(* Prints each command line arg, suffixed with a space. *)  
  
val _ =  
  let  
    val args = CommandLine.arguments()  
  in  
    map (print o concat_space) (reverse args);  
    print "\n"  
  end;
```

- Scheme por Guy L. Steele y Gerald Jay Sussman en 1975.
- SQL (Structured Query Language) en 1978.
- Ada por Jean Ichbiah et al. en 1983.
- C++ por Bjarne Stroustrup en 1983.
- Common Lisp en 1984.
- Eiffel, Erlang, Perl, Tcl y Fl a finales de los 80's. Ejemplo Erlang:

```
sort([Pivot|T]) ->  
  sort([ X || X <- T, X < Pivot]) ++  
  [Pivot] ++  
  sort([ X || X <- T, X >= Pivot]);  
sort([]) -> [].
```

- Haskell (en honor a Haskell Curry) en 1990.

```

main = do
  forM_ [1..3] $ \i -> do
    print i

  forM_ [7..9] $ \j -> do
    print j

  withBreak $ \break ->
    forM_ [1..] $ \- -> do
      p "loop"
      break ()

  where
    withBreak = ('runContT' return) . callCC
    p = liftIO . putStrLn

```

- Python, Lua, Java, Delphi, JavaScript, PHP, Rebol, Visual Basic, Mozart, entre otros durante los años 90's.
- C#, .NET, J#, Scala, Factor, entre otros apartir del año 2000.

Los principales paradigmas de programación son:

- Declarativos (Funcional, Lógico, Por Restricciones)
- Imperativo
- Relacional
- Orientado a Objetos
- Por Restricciones
- Concurrente
- Orientado a aspectos
- Orientado a agentes

- Una operación es declarativa si siempre que es llamada con los mismos argumentos retorna el mismo resultado.
- Una operación declarativa es:
 - **Independiente:** depende solo de sus argumentos
 - **Sin estado:** no hay memoria entre distintos llamados
 - **Determinista:** un llamado con los mismos argumentos da siempre el mismo resultado, sin importar desde el lugar que se haga.
- Ejemplo: HTML, XML, CSS, Mercury, Prolog.

HTML

```

```

SQL

```
SELECT * FROM Users WHERE Country='Mexico';
```

XML

```
<article>  
  <header>  
    <title>Programación declarativa</title>  
    <text>Solo escribe sin preocuparte de más :)</text>  
  </header>  
</article>
```

- Basado en el cálculo λ .
- Manejo implícito de la memoria.
- El concepto de función es fundamental.
- Funciones son ciudadanos de primera clase (las funciones pueden ser parámetros o valores de retorno de otras funciones).
- Programa: Conjunto de funciones + Aplicación.
- Ejemplos: Lisp, Haskell, Scheme, ML.

Cálculo λ

- Diseñado para investigar la definición de función, la noción de aplicación de funciones y la recursión.
- Utilizado para definir algoritmos computables o decidibles.
- Es una estrategia para definir si un algoritmo es computable, ya que se ha demostrado que el problema de la parada es un problema indecidible.
- Cualquier función computable puede ser expresada y evaluada a través de este cálculo.
- Las funciones son consideradas un valor tipo procedimiento.

- Basado en el cálculo de predicados.
- Mecanismo de demostración automática de teoremas.
- Esencial: Concepto de deducción lógica.
- Programa: Conjunto de axiomas y un objetivo.
- Ejemplos: Prolog.

- Orientado por la máquina.
- Alto nivel.
- Esencial: Asignación y secuenciación.
- La programación está dada en términos del estado del programa.
- Programa: Secuencia de instrucciones.
- Ejemplos: Fortran, Algol, Basic, C, Pascal.

JavaScript

```
class Number {  
  constructor (number = 0) {  
    this.number = number;  
  }  
  
  add (x) {  
    this.number = this.number + x;  
  }  
}  
const myNumber = new Number (5);  
myNumber.add (3);  
alert (myNumber.number); // 8
```

- Se representa el mundo real mediante objetos y sus interacciones.
- Basado en el concepto computacional de objeto.
- Esencial: Concepto de objeto, herencia, mensaje.
- Programa: Conjunto de objetos y sus interacciones.
- Ejemplos: Smalltalk, Java, C++, Obliq, etc.

- Basado en la teoría de concurrencia y cálculos de procesos (Cálculo π , CCS, CCP).
- Esencial: Mecanismos de comunicación entre procesos.
- Programa: Conjunto de procesos.
- Ejemplos: PICT, MWB, Erlang.
- Este paradigma funciona bien en lenguajes funcionales, ya que no se requiere sincronización (semáforos).

Ejemplo en Erlang

```
-module(ejemploFLP).  
  
-export([iniciar/0, dialogo/2]).  
  
dialogo(Entrada, 0) ->  
    done;  
dialogo(Entrada, Contador) ->  
    io:format("~p~n", [Entrada]),  
    dialogo(Entrada, Contador - 1).  
  
iniciar() ->  
    spawn(ejemploFLP, dialogo, [hola, 5]),  
    spawn(ejemploFLP, dialogo, [adios, 4]).  
  
%Funcionamiento  
%c(ejemploFLP).  
%ejemploFLP:iniciar().
```

- Las funciones pueden tener cero, una o más salidas (Frente a funciones)
- Puede intercambiarse el rol de las entradas y salidas
- Selección no-determinista de una opción entre varias
- Ejemplo: Prolog (Búsqueda sobre una base de conocimiento), Analizadores sintácticos, Bases de datos relacionales (SQL).

Ejemplo de paradigma relacional con Prolog.

```
%Base del conocimiento
amigo(juan , pedro).
amigo(juan , carlos).
amigo(pedro , maria).

%consultas
%amigo(juan , pedro). %Retorna yes
%amigo(juan , X). %Retorna X= pedro y X = carlos
```

Observe que

- 1 Hay una búsqueda en un conjunto de datos
- 2 Se retornan cero o más valores en una consulta.

SQL también es un lenguaje relacional de búsqueda sobre bases de datos.

- Basado en el concepto de restricción (un predicado o relación lógica).
- **Esencial:** Concepto de consecuencia lógica.
- **Esencial:** Búsqueda en arboles y reducción de dominios (distribución y propagación).
- **Programa:** Variables + Restricciones (Conjunto de Relaciones entre variables) + Estrategia de exploración.
- La búsqueda de soluciones es concurrente (se exploran varias posibilidades a la vez).
- **Ejemplos:** CLP, Mozart.

Ejemplo de paradigma de programación con restricciones con Mozart OZ. Problema, encontrar los valores de las letras para que $SEND + MORE = MONEY$.

```
declare
proc {Money Root}
local
  S E N D M O R Y
in
  Root = sol(s:S e:E n:N d:D m:M o:O r:R y:Y)           % Registro con letras
  Root ::: 0#9                                           % Dominio de búsqueda de
    las variables entre 0 y 9

  %Restricciones
  {FD.distinct Root}

  %S y M distintos de 0 (si no todo se hace 0)
  S \=: 0
  M \=: 0

  1000*S + 100*E + 10*N + D
+
  1000*M + 100*O + 10*R + E
=: 10000*M + 1000*O + 100*N + 10*E + Y

  %Estrategia de búsqueda (probar primero con números pequeños del dominio)
  {FD.distribute ff Root}
end
%Explore todas las posibilidades
{ExploreAll Money}
```

- Modularidad de las aplicaciones y separación de conceptos (generalmente conceptos técnicos y comunes a toda la aplicación).
- Separación de las funcionalidades comunes utilizadas en la aplicación de las funcionalidades propias de cada módulo.
- Esencial: Concepto de aspecto (funcionalidad transversal).
- Ejemplos: AspectJ, Aspect, phpAspect, Aspysct AOP.

- Se representa el mundo real mediante agentes y sus interacciones a través de mensajes.
- Basado en el concepto de agentes.
- Un agente es una entidad computacional situada en algún entorno y que es capaz de ejecutar acciones autónomas en dicho entorno con el fin de cumplir sus objetivos de diseño.
- Hilo de ejecución independiente, comunicación por paso de mensajes, conocimiento parcial del entorno, mecanismo de toma de decisiones, reactividad, proactividad, habilidad social.
- Programa: Conjunto de agentes y sus interacciones.
- Ejemplos: JADE, JASON.

- Enfoques basados en componentes.
- Mecanismos de seguridad y de confiabilidad, seguridad en hilos.
- Énfasis en movilidad y distribución. Enfoques basados en paradigmas y tecnologías actuales (computación grid y cloud, Map and Reduce).

- Incrementa la capacidad para expresar ideas.
- Amplía el espectro de conocimientos necesario para seleccionar un lenguaje.
- Incrementa la habilidad para aprender nuevos lenguajes y paradigmas.

- Mejor entendimiento de como los lenguajes de programación están implementados.
- Mejor uso de los lenguajes de programación que ya se conocen.
- Progreso global de las ciencias computacionales.